Python: exercises on files

This notebook was generated in student mode.

Exercise 1: Writing and Reading a File

Exercise Description

Write a program that:

1. Creates and opens a file named `"students.txt"` in write mode.
2. Writes the names of five students, each on a new line, to the file.
3. Closes the file.
4. Reopens the file in read mode and prints each student's name.

Store the result in a variable `result` containing a list of the student names read from the file.

Your solution

```
# Write your code here
result = []
```

Test your solution

```
# Test that result contains the expected student names
assert len(result) == 5
assert "Alice" in result
assert "Bob" in result
assert "Charlie" in result
assert "David" in result
assert "Eve" in result
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 2: Storing and Loading Scores

Exercise Description

1. Create a list of five scores: `[85, 90, 78, 92, 88]`.
2. Write a program that:
    - Opens a file named `"scores.txt"` in write mode.
    - Writes each score from the list to the file, with each score on a new line.
    - Closes the file.
3. Reopen the file in read mode and:
    - Read all lines from the file, convert each line to an integer, and store the results in a new list.
    - Store the loaded scores in a variable `loaded_scores`.

Your solution

```
# Write your code here
scores = [85, 90, 78, 92, 88]
loaded_scores = []
```

Test your solution

```
# Test that loaded_scores matches the original scores
assert loaded_scores == [85, 90, 78, 92, 88]
assert len(loaded_scores) == 5
assert all(isinstance(score, int) for score in loaded_scores)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 3: Appending to a File

Exercise Description

Write a program that:

1. Opens a file named `"notes.txt"` in append mode.
2. Adds three predefined notes to the file: "First note", "Second note", "Third note".
3. Each note should be written on a new line.
4. Finally, opens the file in read mode and stores all notes in a variable `all_notes` as a list.

Store the result in a variable `all_notes` containing a list of all the notes read from the file.

Your solution

```
# Write your code here
all_notes = []
```

Test your solution

```
# Test that all_notes contains the expected notes
assert "First note" in all_notes
assert "Second note" in all_notes
assert "Third note" in all_notes
assert len(all_notes) >= 3  # May contain more if file existed before
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 4: Counting Words in a File

Exercise Description

Write a program that:

1. Uses the predefined sentence: "Python is a powerful programming language".
2. Opens a file named `"sentence.txt"` in write mode and writes the sentence to the file.
3. Closes the file.
4. Reopens the file in read mode, reads the sentence back, and counts the number of words.
5. Stores the word count in a variable `word_count`.

Hint: Use `split()` on the sentence to count the words.

Your solution

```
# Write your code here
sentence = "Python is a powerful programming language"
word_count = 0
```

Test your solution

```
# Test that word_count is correct
assert word_count == 6  # "Python is a powerful programming language" has 6 words
assert isinstance(word_count, int)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 5: Saving and Loading a Shopping List

Exercise Description

Write a program that:

1. Uses a predefined shopping list: `["apple", "bread", "milk"]`.
2. Opens a file named `"shopping_list.txt"` in write mode and writes each item from the list to a new line.
3. Reopens the file in read mode and:
    - Reads all lines into a list.
    - Stores the shopping list from the file in a variable `loaded_shopping_list`.

Your solution

```
# Write your code here
shopping_list = ["apple", "bread", "milk"]
loaded_shopping_list = []
```

Test your solution

```
# Test that loaded_shopping_list matches the original
assert loaded_shopping_list == ["apple", "bread", "milk"]
assert len(loaded_shopping_list) == 3
assert "apple" in loaded_shopping_list
assert "bread" in loaded_shopping_list
assert "milk" in loaded_shopping_list
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 6: Saving Calculated Results to a File

Exercise Description

1. Create a list of numbers: `[5, 10, 15, 20]`.
2. Write a program that:
    - Calculates the square of each number and writes the results to a file named `"squares.txt"`.
    - Each square should be written on a new line.
3. Open the file in read mode and store each square in a variable `squares_list` as a list of integers.

Your solution

```
# Write your code here
numbers = [5, 10, 15, 20]
squares_list = []
```

Test your solution

```
# Test that squares_list contains the correct squares
assert squares_list == [25, 100, 225, 400]  # 5^2, 10^2, 15^2, 20^2
assert len(squares_list) == 4
assert all(isinstance(square, int) for square in squares_list)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 7: Reversing Lines in a File

Exercise Description

Write a program that:

1. Uses five predefined lines of text: `["Line 1", "Line 2", "Line 3", "Line 4", "Line 5"]`.
2. Saves each line to a file named `"reversed_lines.txt"`.
3. Opens the file in read mode and prints each line in reverse order.
4. Stores the reversed lines in a variable `reversed_lines` as a list.

Your solution

```
# Write your code here
lines = ["Line 1", "Line 2", "Line 3", "Line 4", "Line 5"]
reversed_lines = []
```

Test your solution

```
# Test that reversed_lines contains lines in reverse order
assert reversed_lines == ["Line 5", "Line 4", "Line 3", "Line 2", "Line 1"]
```

```
assert len(reversed_lines) == 5
assert reversed_lines[0] == "Line 5"
assert reversed_lines[-1] == "Line 1"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 8: Finding the Longest Word in a File

Exercise Description

Write a program that:

1. Uses the predefined sentence: "Programming with Python is incredibly rewarding".
2. Writes the sentence to a file named `"sentence.txt"`.
3. Opens the file in read mode, reads the sentence back, and finds the longest word.
4. Stores the longest word in a variable `longest_word`.

Your solution

```
# Write your code here
sentence = "Programming with Python is incredibly rewarding"
longest_word = ""
```

Test your solution

```
# Test that longest_word is correct
assert longest_word == "Programming"  # "Programming" is the longest word (11 characters)
assert len(longest_word) == 11
assert isinstance(longest_word, str)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 9: Reading Data and Calculating the Average

Exercise Description

1. Create a file named `"data.txt"` containing numerical values: `[10, 15, 20, 25, 30]`, each on a new line.
2. Write a program that:
    - Reads each value from the file.
    - Calculates the average of the values.
3. Store the calculated average in a variable `average`.

Your solution

```
# Write your code here
data_values = [10, 15, 20, 25, 30]
average = 0
```

Test your solution

```
# Test that average is calculated correctly
assert average == 20.0  # (10+15+20+25+30)/5 = 100/5 = 20.0
assert isinstance(average, float)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 10: Saving a List of Names in Uppercase

Exercise Description

Write a program that:

1. Uses five predefined names: `["alice", "bob", "charlie", "diana", "eve"]`.
2. Saves each name in uppercase to a file named `"uppercase_names.txt"`, with each name on a new line.
3. Opens the file in read mode and stores each uppercase name in a variable `uppercase_names` as a list.

Your solution

```
# Write your code here
names = ["alice", "bob", "charlie", "diana", "eve"]
uppercase_names = []
```

Test your solution

```python
# Test that uppercase_names contains the correct uppercase names
assert uppercase_names == ["ALICE", "BOB", "CHARLIE", "DIANA", "EVE"]
assert len(uppercase_names) == 5
assert all(name.isupper() for name in uppercase_names)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 11: Writing and Checking for Palindromes

Exercise Description

Write a program that:

1. Uses five predefined words: `["racecar", "hello", "level", "python", "radar"]`.
2. Saves each word to a file named `"words.txt"`.
3. Reopens the file, reads each word, and stores only those words that are palindromes (words that read the same forwards and backwards) in a variable `palindromes` as a list.

Your solution

```
# Write your code here
words = ["racecar", "hello", "level", "python", "radar"]
palindromes = []
```

Test your solution

```
# Test that palindromes contains only the palindromic words
assert set(palindromes) == {"racecar", "level", "radar"}  # Only palindromes
assert len(palindromes) == 3
assert "hello" not in palindromes  # Not a palindrome
assert "python" not in palindromes  # Not a palindrome
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 12: Counting Specific Words in a Text File

Exercise Description

1. Create a file named `"story.txt"` containing a short paragraph with predefined text.
2. Write a program that:
    - Uses the word "the" as the target word to count.
    - Reads the contents of the file and counts how many times the word appears in the text (case-insensitive).
3. Store the total count in a variable `word_count`.

Your solution

```
# Write your code here
story_text = ["Once upon a time in a land far, far away there was a brave knight.",
              "The knight set out on an adventure to rescue the kingdom."]
word_to_count = "the"
word_count = 0
```

Test your solution

```
# Test that word_count is correct
assert word_count == 2  # "the" appears twice in the story ("The knight" and "the kingdom")
assert isinstance(word_count, int)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pythontutor.
```

Exercise 13: Writing a Simple CSV File

Exercise Description

Write a program that: